Nome: Lucas Henrique Motta RA: 2669730
Nome: Dirceu Morais da Costa Júnior RA: 2669455

Atividade Avaliativa A2 - Parte 2

1. Projeto Inicial

História selecionada: 1 - Reservar atendimento de monitoria.

Classes e sua função

1. **Materia:** representa uma disciplina. Guarda o nome. Serve para filtrar horários.
2. **Aluno:** representa quem reserva. Guarda RA e nome. Não faz lógica de reservar.
3. **Monitor:** representa quem atende. Guarda RA, nome e a disciplina que atende, responsavel pelos horários que oferece.
4. **Horario:** representa o atendimento de 30 minutos. Guarda monitor, disciplina, data/hora de início, capacidade (1 = individual, maior que 1 = grupo) e a lista de reservas ativas. Sabe informar se está livre e quantas vagas restam.
5. **Reserva:** representa a ligação entre um aluno e um horário. Guarda aluno, sala, horário e situação (ativa, cancelada, falta, realizada). Nesta história só é criada como "ativa".
6. **ServicoReserva:** concentra as regras da história. Mantém a lista de horários e reservas, e oferece:
    a. listar os horários livres de uma disciplina;
    b. reservar um horário para um aluno, validando existência, capacidade e limite semanal;

Erros próprios

HorarioInexistenteError, HorarioLotadoError e LimiteSemanalError.

Critérios e testes

Critério	Teste	Resultado
Lista só horários livres da disciplina	`test_lista_apenas_horarios_livres_da_disciplina`	sucesso
Disciplina sem horários livres	`test_lista_vazia_quando_nao_ha_horarios_livres`	falha/limite

Critério	Teste	Resultado
Reserva registrada e vaga ocupada	`test_reserva_registra_e_ocupa_vaga`	sucesso
Horário lotado é recusado	`test_reserva_em_horario_lotado_lanca_erro`	falha
Horário em grupo aceita até a capacidade	`test_horario_em_grupo_aceita_ate_a_capacidade`	sucesso
Limite semanal excedido é recusado	`test_limite_semanal_excedido_lanca_erro`	falha
Semana seguinte não conta no limite	`test_reserva_em_outra_semana_nao_conta_no_limite`	sucesso
Horário inexistente é recusado	`test_reserva_em_horario_inexistente_lanca_erro`	falha

Registro de mudanças

- UML substituído por projeto textual. O plano da Parte 1 previa UML, mas o enunciado exige projeto inicial sem diagramas.
- Classes acrescentadas: Horario, ServicoReserva e erros.py (exceções próprias). As regras de capacidade e de limite semanal precisavam de um lugar que os testes chamassem direto, sem entrada digitada.
- Fora do escopo: Professor, Falta, senha e login. A História 1 não os usa; as histórias 2 a 5 foram apenas especificadas.
- ReservaDuplicadaError acrescentado. Sem ele, o mesmo aluno poderia ocupar duas vagas de um mesmo horário em grupo.
- Testes além da tabela do projeto inicial: limite semanal por disciplina, limite inválido, reserva duplicada e testes de unidade de Horario, Reserva e erros.
- Premissas (cliente não decidiu): limite de 2 reservas por semana por disciplina, configurável; capacidade do horário definida pelo monitor.

2. Relatório de Execução

Como executar

- Python: 3.8 ou superior (testado com 3.15 e 3.14.7).
- Bibliotecas: nenhuma além da biblioteca padrão.
- Arquivo principal: “`codigo/main.py`”. Comando: “`python codigo/main.py`”
- Testes (na raiz do repositório): “`python -m unittest discover -s codigo -v`”

Plano x realizado

Etapa do plano	**Data planejada**	**Realizado (responsável)**
História de usuário	19/10	Lucas
projeto inicial	19/10	Dirceu
implementação de classe pt1	20-21/10	Dirceu
implementação de classe pt2	20-21/10	Lucas
Testes automatizados	20-21/10	Lucas
Documentação	22/10	Juntos

Mudanças em relação ao plano e ao projeto inicial

7. UML substituído por projeto textual.** O plano da Parte 1 previa UML, mas o enunciado exige projeto inicial sem diagramas.
8. Classes acrescentadas: “`Horario`”, “`ServicoReserva`” e “`erros.py`” (exceções próprias). As regras de capacidade e de limite semanal precisavam de um lugar que os testes chamassem direto, sem entrada digitada.
9. Fora do escopo: “Professor”, “Falta”, senha e login. A História 1 não os usa; as histórias 2 a 5 foram apenas especificadas.
10. “`ReservaDuplicadaError`” acrescentado. Sem ele, o mesmo aluno poderia ocupar duas vagas de um mesmo horário em grupo.
11. Testes além da tabela do projeto inicial: limite semanal por disciplina, limite inválido, reserva duplicada e testes de unidade de “`Horario`”, “`Reserva`” e erros.
12. Premissas (cliente não decidiu): limite de 2 reservas por semana por disciplina, configurável; capacidade do horário definida pelo monitor.

Histórias de Usuário
Sistema de agendamento de monitorias

Premissas assumidas (cliente ainda não decidiu):

- Cada atendimento dura 30 minutos.
- Limite de 2 reservas por semana por disciplina, configurável.
- Cada horário tem uma capacidade (1 = individual; maior que 1 = atendimento em grupo), definida pelo monitor.
- Faltas não bloqueiam o aluno automaticamente; só geram aviso após um limite configurável (por padrão: 3).

História 1: reservar atendimento de monitoria

Como aluno, quero escolher uma disciplina, ver os horários livres e reservar um atendimento de 30 minutos, para não precisar combinar horário por mensagem com o monitor.

critérios de aceitação:

- Quando escolher uma disciplina, o sistema lista apenas os horários livres dessa disciplina.
- Ao reservar um horário livre, a reserva é registrada e uma vaga do horário é ocupada.
- Se o horário já atingiu a capacidade, a reserva é recusada com erro.
- Se o aluno já atingiu o limite semanal de reservas na disciplina, a nova reserva é recusada.
- Horários reservados não são exibidos como possibilidade de reserva na tabela de horários.

prioridade: Alta. É basicamente a principal ideia de implementação vinda do email.

trecho do e-mail: "A ideia é que o aluno escolha a disciplina, veja os horários livres e reserve um atendimento."

História 2: cancelar atendimento e avisar os alunos

Como monitor, quero que ao cancelar um horário de atendimento, os alunos com reserva sejam avisados, para que ninguém chegue na sala e encontre o atendimento cancelado.

critérios de aceitação:

- Ao cancelar um horário, ele passa para a situação "cancelado" e deixa de aparecer como livre.
- Todas as reservas desse horário passam para a situação "cancelada".
- Cada aluno com reserva afetada recebe uma notificação registrada, com disciplina, data e hora.

- Um monitor só pode cancelar horários da própria disciplina; caso contrário, o cancelamento é recusado.

prioridade: Alta. Os alunos só descobrem o cancelamento ao chegar, gerando perda de tempo.

trecho do e-mail: "[...] às vezes precisa cancelar por causa de prova. Os alunos reclamam que só descobrem o cancelamento quando chegam na sala."

História 3: controlar faltas dos alunos

Como monitor, quero registrar quando um aluno reserva e não comparece, para ter o controle das faltas e avisar o aluno e a coordenação quando forem excessivas.

critérios de aceitação:

- O monitor só pode registrar falta em reservas da sua disciplina cujo horário já passou.
- A falta registrada guarda aluno, disciplina, horário e, opcionalmente, o motivo.
- A reserva faltada passa para a situação "falta" e o contador de faltas do aluno aumenta em 1.
- Quando o contador atinge o limite configurado, são gerados avisos para o aluno e para a coordenação, sem bloquear novas reservas.

Prioridade: Média. O horário perdido é um problema, mas o cliente não decidiu a regra de bloqueio e a reserva funcionando já resolve a maior parte.

Trecho do e-mail: "Quando o aluno reserva e não aparece, o horário fica perdido. Alguns monitores sugeriram bloquear quem falta muito, mas outros acham isso exagerado."

História 4: ver os conteúdos que mais geram dúvidas

Como professor, quero consultar os assuntos mais citados nos atendimentos da disciplina, pra reforçar esses conteúdos em aula.

critérios de aceitação:

- Após um atendimento realizado, o monitor registra o assunto e um resumo das dúvidas.
- O professor visualiza a lista de assuntos da sua disciplina ordenada do mais para o menos frequente.
- O professor não vê dados de disciplinas de outros professores.
- Se não existir atendimentos com assunto registrado, o sistema informa que não há dados.

prioridade: Baixa. É útil pro ensino, mas não é necessária para o funcionamento das reservas.

trecho do e-mail: "Os professores das disciplinas gostariam de saber quais conteúdos geram mais dúvidas, para reforçar em aula."

História 5: relatório mensal de horas dos monitores

Como coordenador,quero gerar um relatório mensal com as horas atendidas por cada monitor, para calcular o pagamento das bolsas.

critérios de aceitação:

- O relatório lista, para o mês escolhido, as horas atendidas por cada monitor.
- Só contam atendimentos realizados; horários cancelados e sem nenhum aluno presente não contam.
- Atendimento em grupo conta uma única vez, e não uma vez por aluno.
- Apenas o coordenador acessa o relatório; alunos, monitores e professores recebem acesso negado.

prioridade: Média. A coordenação precisa dele todo mês, mas depende de existirem atendimentos registrados.

trecho do e-mail: "A coordenação precisa de um relatório mensal com as horas atendidas por cada monitor, porque isso conta para o pagamento da bolsa. Essas informações de bolsa não podem ficar visíveis para todo mundo."

Testes de Aceitação

História implementada: **História 1: Reservar atendimento de monitoria (**localizada dentro de `docs/`). Os testes ficam em `codigo/testes/test_servico_reserva.py` (classe `TestReservarAtendimento`) e chamam o `ServicoReserva` direto, sem entrada digitada.

Comando: `python -m unittest discover -s codigo -v`.

Critérios x testes

Critério de aceitação	Teste	Cenário
1. Lista só os horários livres da disciplina escolhida	`test_lista_apenas_horarios_livres_da_disciplina`	Sucesso
	`test_lista_vazia_quando_nao_ha_horarios_livres`	Limite (lista vazia)

2. Reserva registrada e vaga ocupada	`test_reserva_registra_e_ocupa_vaga`	Sucesso
3. Horário lotado é recusado com erro	`test_reserva_em_horario_lotado_lanca_erro`	Falha/exceção
	`test_horario_em_grupo_aceita_ate_a_capacidade`	Sucesso
4. Limite semanal excedido é recusado	`test_limite_semanal_excedido_lanca_erro`	Falha/exceção
	`test_reserva_em_outra_semana_nao_conta_no_limite`	Sucesso
	`test_limite_semanal_e_por_disciplina`	Sucesso
5. Horário inexistente é recusado	`test_reserva_em_horario_inexistente_lanca_erro`	Falha/exceção

O critério 2 tem apenas teste de sucesso. Os cenários de falha são os dos critérios 3, 4 e 5, que são justamente os motivos de recusa de uma reserva.

Testes fora dos critérios

- `test_reserva_duplicada_lanca_erro` e `test_limite_semanal_invalido_lanca_erro` (falha/exceção): regras acrescentadas durante a implementação.
- `test_horario.py`, `test_reserva.py` e `test_erros.py`: testes de unidade de `Horario`, `Reserva` e das exceções.

O que não foi coberto, e por quê

- **`main.py`:** é só a interface de terminal e foi verificado executando manualmente. A lógica testada está nas classes.
- **Horários no passado:** o critério não pede isso, então a listagem não aplica.
- **Persistência:** os dados ficam em memória, e o protótipo não grava nada no disco.
- **Classes Aluno, Monitor e Materia reais:** os testes usam objetos simples, para não depender de classes sem regra de negócio nesta história.